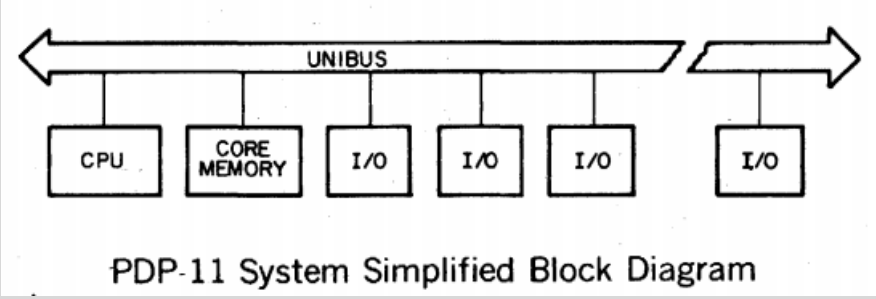

INTRODUCTION TO IO & DRIVERS

Components of I/O Subsystem

- I/O Hardware
  - ports, buses, devices, controllers
- I/O Software
  - Interrupt Handlers, Device Driver,
  - Device-Independent Software,
  - User-Space I/O Software
- I/O Data transfer mechanisms
  - Polling, Interrupt and DMAs

Older System Architecture



PDP-11 System Simplified Block Diagram

- Peripherals communicate via the UNIBUS
  → All devices (CPU, memory, I/O) share a single system bus.
- Peripherals are memory-mapped
  → Each device is assigned a specific memory address range it responds to.
- No memory protection
  → Any program can access any memory location, including device registers.

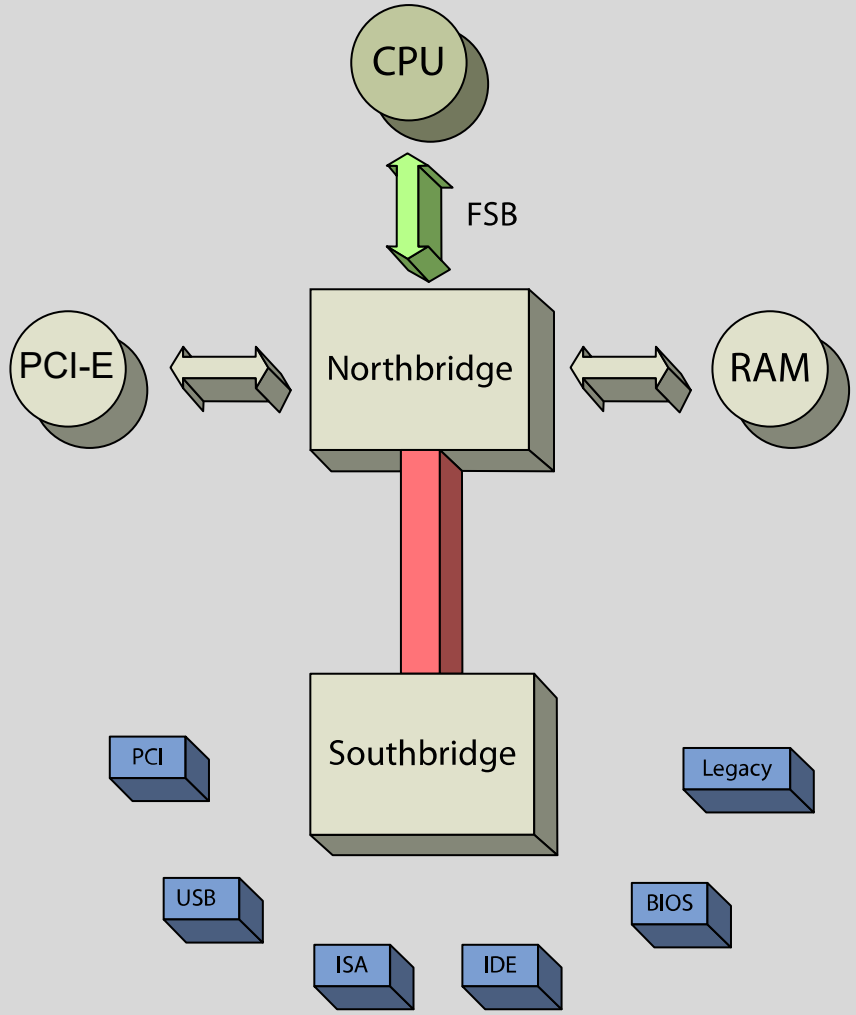
Older System Architecture

- Every individual device had its own unique hardware interface
  → No standardization, even between devices of the same type.
- Applications included custom code for each specific device
  → Programs were tightly coupled to the hardware they ran on.
- Applications directly polled devices to send or receive data
  → Constantly checked device status instead of using interrupts.

Modern Device Interfaces

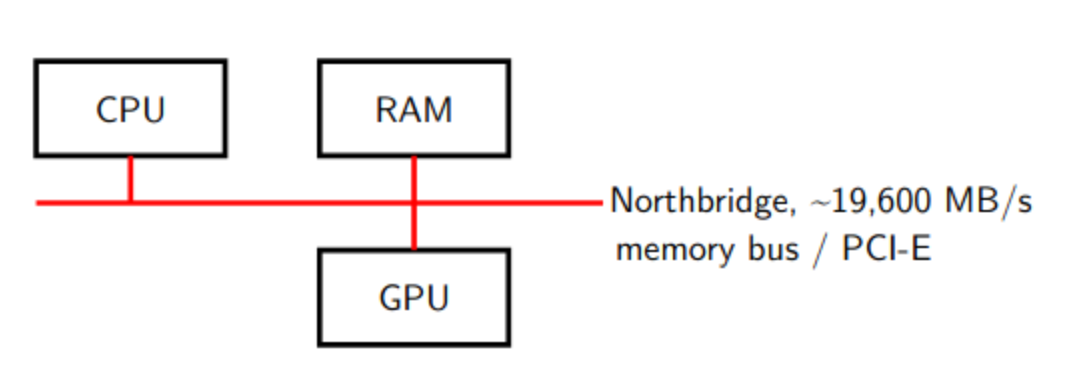
- The operating system manages all device communication and access
  - Applications no longer interact with hardware directly.
- A consistent, standardized interface is provided to applications
  - Programs use the same API regardless of the underlying device.
- I/O operations are interrupt-driven
  - Devices signal the OS when they’re ready, avoiding constant polling.

A typical north/southbridge layout



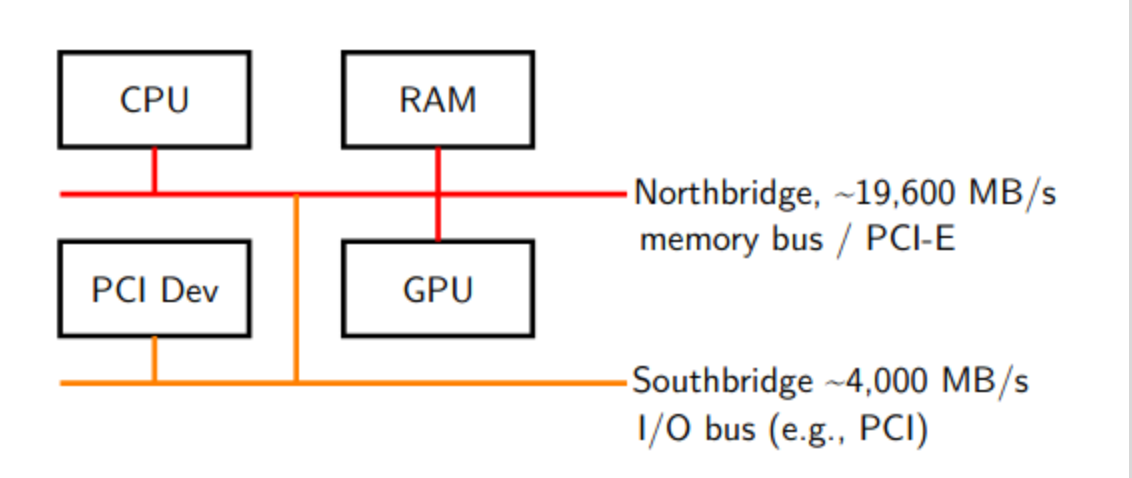
https://en.wikipedia.org/wiki/Southbridge_%28computing%29

Hardware support for devices: Northbridge



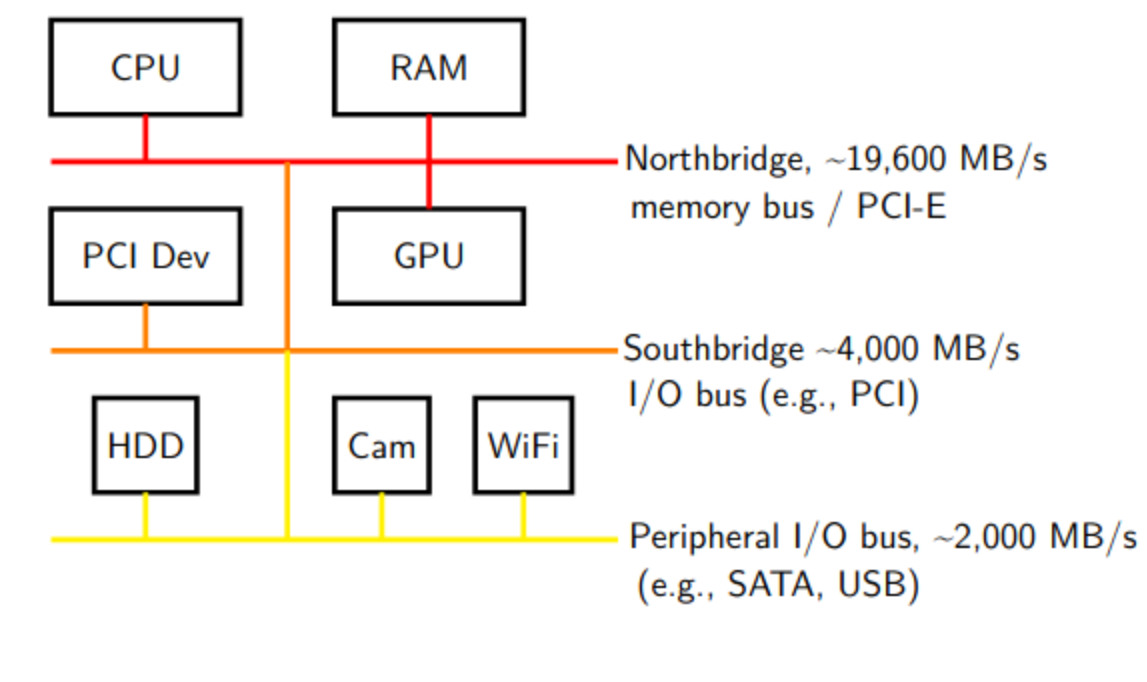
What about other devices?

Hardware support for devices: Southbridge



What about "slow" IO?

Hardware support for devices: Other IO

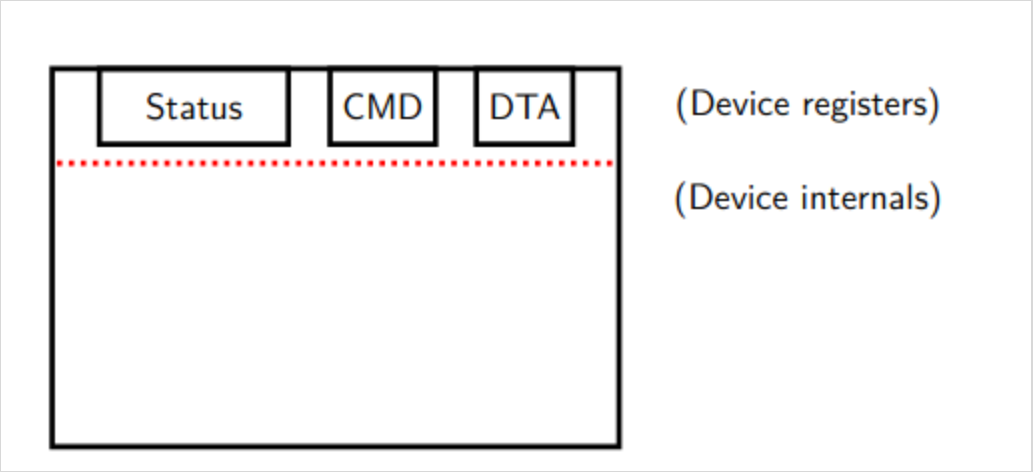


Terminology

- The northbridge is also known as the Memory Controller Hub (MCH).
- The southbridge is referred to as the I/O Controller Hub (ICH) in Intel systems or the Fusion Controller Hub (FCH) in AMD systems.
- The southbridge connects to the CPU indirectly through the northbridge, which maintains a direct connection to the CPU.

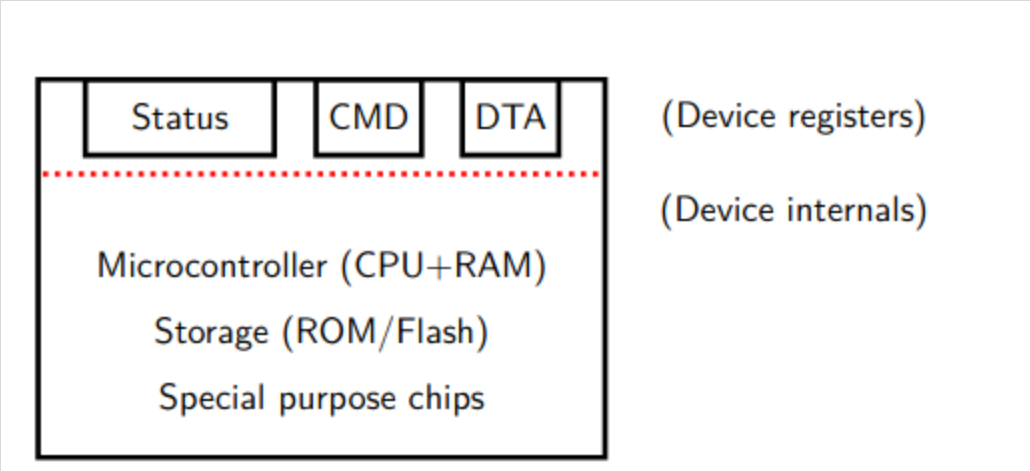
- But how do devices work

Canonical Devices



- OS writes device registers (by executing CPU instructions)
- Device internals are hidden (abstraction)

OS-Device Communication via Drivers



- The operating system communicates with hardware devices using an agreed-upon protocol implemented by device drivers.
- Device drivers act as intermediaries, translating OS requests into device-specific commands.
- Devices notify the OS about events or status changes through shared memory signals or by generating interrupts that prompt immediate attention.

Device Protocol - Polling

- The CPU repeatedly checks a device to see if it needs attention or has data ready
- The CPU asks the device status at regular intervals, like "Are you ready yet?"
- It is a busy-wait approach, meaning the CPU is actively waiting and not doing other work
- When is Polling Used?
  - Useful for simple or fast devices where waiting times are short
  - Works well when data arrives in short bursts or at predictable intervals

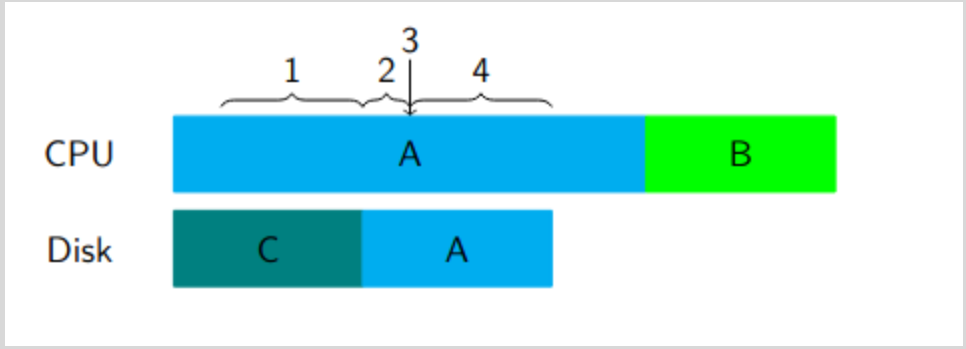
Device Protocol

```
while (STATUS == BUSY)  ; // 1. spin

// 2. Write data to DTA register
*dtaRegister = DATA;

// 3. Write command to CMD register
*cmdRegister = COMMAND;

while (STATUS == BUSY) ; // 4. spin
```

- **Step 1:** Spin-wait until the device status shows it is ready.
- **Step 2:** Write the required data into the data register.
- **Step 3:** Write the command to the command register to start the operation.
- **Step 4:** Spin-wait again until the device finishes processing the command.

- Are there any problems?

Device protocol

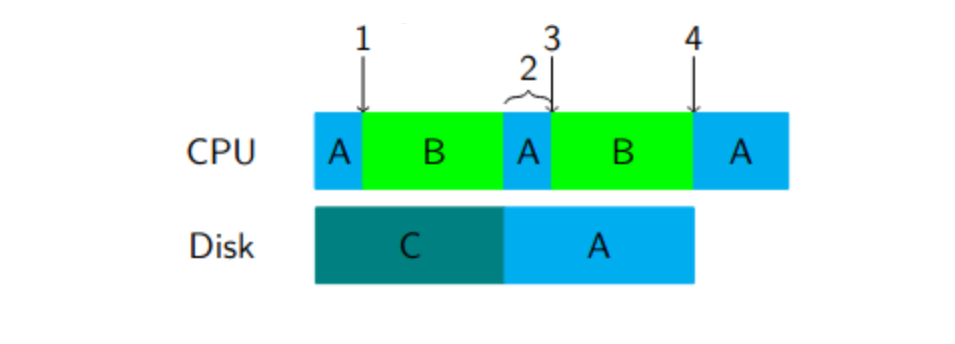


- Busy waiting (1. and 4.) wastes cycles twice
- CPU should not need to wait for completion of command
- Solution: embrace asynchronous communication
  - Inform device of request (to get rid of 1.)
  - Wait for signal of completion (to get rid of 4.)

Asynchronous - Interrupt

- What are Interrupts?
  - Devices send a signal to the CPU when they need attention
  - The CPU can do other work until it is interrupted
  - When interrupted, the CPU pauses its current task to handle the device request
- When are Interrupts Used?
  - Ideal for slow or unpredictable devices
  - Useful when waiting for data that arrives irregularly

Device protocol



- Instead of spinning, the OS (driver) waits for an interrupt
  - Interrupts are handled centrally through a dispatcher
  - On interrupt arrival, wake up kernel thread that waits on that interrupt

Interrupt Performance

- Can interrupts lead to worse performance than polling?

- Yes, due to livelock:
  - The system stays busy handling interrupts but never makes real progress, causing starvation.

- Analogy:
  - Like a waiter constantly interrupted by customers' questions, never actually serving food.

- Example:
  - A flood of network packets triggers frequent interrupts and context switches, preventing the application from processing them, so packets queue up.

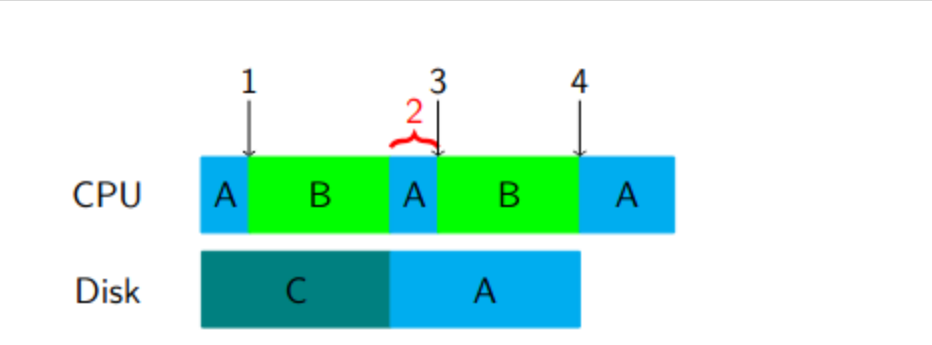
Balancing Polling and Interrupts in Real Systems

- Real systems combine polling and interrupts for efficiency.
- Interrupts are ideal for slow devices because they allow computation and I/O to overlap.
- Polling works better for short bursts or small data transfers because it has less overhead.
- An optimization called coalescing means devices wait briefly to batch multiple requests before sending, reducing interrupt frequency and improving performance.

Next

- Programmed I/O
- DMA

Introduction to Programmed I/O (PIO)



- CPU controls all data transfer by directly reading/writing device registers
- Data is transferred one byte or word at a time with CPU instructions
- CPU waits (busy-waits) during the entire transfer process
- Simple to implement but CPU is heavily involved, limiting multitasking
- Efficient only for small amounts of data or fast devices

Programmed I/O

```
#define STATUS_REGISTER (*(volatile int*)0x1000)
#define DATA_REGISTER   (*(volatile char*)0x1004)
#define DEVICE_READY 1


void readDataPIO(char* buffer, int length) {
   for (int i = 0; i < length; i++) {
      while ((STATUS_REGISTER & DEVICE_READY) == 0) {
         // Busy-wait (polling)
      }
      buffer[i] = DATA_REGISTER;
   }
}
```

- **STATUS_REGISTER** is a device register that tells us if the device is ready to send data.
- The CPU keeps checking this register in a loop (busy-wait) until the device signals readiness.
- When ready, the CPU reads one byte from **DATA_REGISTER**.
- This process repeats for every byte, meaning the CPU is occupied during the entire data transfer, which wastes CPU cycles.
- This approach works for small or fast data transfers but becomes inefficient with large data or slow devices.

Thank You